Poseidon Circuit

목적

Poseidon Circuit은 **Poseidon 해시 함수의 계산을 검증**하는 회로입니다. Poseidon은 SNARK-friendly 해시 함수로, zkTrie (Sparse Binary Merkle Patricia Trie)와 바이트코드 해싱에 사용됩니다. 이 회로는 MPT Circuit과 Bytecode Circuit에서 필요한 Poseidon 해시 계산을 효율적으로 검증합니다.

Poseidon 해시 함수

SNARK-Friendly 해시

Poseidon은 영지식 증명 시스템에 최적화된 해시 함수입니다:

주요 특징:

1. **낮은 회로 복잡도**: 제약 조건 수가 적음
2. **필드 연산 기반**: 유한체(Finite Field) 산술 연산 사용
3. **빠른 증명 생성**: Keccak256 대비 10-100배 빠름
4. **작은 증명 크기**: 효율적인 회로 구조

Poseidon vs Keccak256

| 특징 | Keccak256 | Poseidon |

|------|----------|----------|

| **설계 목적** | 범용 해시 | SNARK 최적화 |

| **연산 타입** | 비트 연산 | 필드 연산 |

| **회로 크기** | 큼 (150k+ 제약) | 작음 (수백 제약) |

| **증명 시간** | 느림 | 빠름 |

| **이더리움 호환** | 네이티브 | 추가 변환 필요 |

회로 구조

PoseidonCircuit 정의

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 16-17)

```
#[derive(Default, Clone, Debug)]
pub struct PoseidonCircuit<F: Field>(
pub(crate) PoseidonHashTable<F>, // 해시 테이블 데이터
usize // max_hashes (최대 해시 수)
);
```

PoseidonCircuitConfig

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 26-27)

```
#[derive(Debug, Clone)]
pub struct PoseidonCircuitConfig<F: Field>(
pub(crate) PoseidonHashConfig<F> // hash-circuit 크레이트의 설정
);
```

이 회로는 외부 `hash-circuit` 크레이트의 `PoseidonHashConfig` 를 래핑합니다.

Poseidon 테이블

PoseidonTable 스키마

파일: `zkevm-circuits/src/table.rs` (line 947-962)

```
pub struct PoseidonTable {
pub q_enable: Column<Fixed>, // 활성화 플래그
pub hash_id: Column<Advice>, // 해시 ID (최종 해시 값)
pub input0: Column<Advice>, // 첫 번째 입력
pub input1: Column<Advice>, // 두 번째 입력
pub control: Column<Advice>, // 제어 플래그 (길이 정보)
pub domain_spec: Column<Advice>, // 도메인 구분자
pub heading_mark: Column<Advice>, // 첫 행 마커
}
```

테이블 파라미터

파일: `zkevm-circuits/src/table.rs` (line 992-996)

```
impl PoseidonTable {
/// Poseidon 순열의 너비
pub(crate) const WIDTH: usize = 3;
/// 입력 너비 (WIDTH - 1)
pub(crate) const INPUT_WIDTH: usize = Self::WIDTH - 1; // = 2
```

```
}
```

설명:

- `WIDTH = 3` : Poseidon 상태는 3개의 필드 원소
- `INPUT_WIDTH = 2` : 한 번에 2개의 입력을 처리
- 하나의 필드 원소는 출력/상태로 사용

해시 블록 크기

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 34)

```
pub const HASH_BLOCK_STEP_SIZE: usize =
HASHBLOCK_BYTES_IN_FIELD * PoseidonTable::INPUT_WIDTH;
```

- `HASHBLOCK_BYTES_IN_FIELD` : 한 필드 원소에 들어가는 바이트 수
- `INPUT_WIDTH = 2` : 두 개의 입력 필드
- 한 번에 처리할 수 있는 바이트 수 결정

회로 설정

SubCircuitConfig 구현

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 36-52)

```
impl<F: Field> SubCircuitConfig<F> for PoseidonCircuitConfig<F> {
type ConfigArgs = PoseidonCircuitConfigArgs;
```

```
fn  new(
meta: &mut ConstraintSystem<F>,
Self::ConfigArgs { poseidon_table }: Self::ConfigArgs,
) -> Self {
// PoseidonTable을 튜플 형태로 변환
let  poseidon_table = (
poseidon_table.q_enable,
[
poseidon_table.hash_id,
poseidon_table.input0,
poseidon_table.input1,
poseidon_table.control,
poseidon_table.domain_spec,
poseidon_table.heading_mark,
],
);
// hash-circuit의 PoseidonHashConfig 생성
let  conf = PoseidonHashConfig::configure_sub(
meta,
poseidon_table,
HASH_BLOCK_STEP_SIZE
);
Self(conf)
```

```
}

}
```

Witness 생성

new_from_block 구현

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 54-91)

```
fn new_from_block(block: &witness::Block) -> Self {

let max_hashes = block.circuits_params.max_poseidon_rows / F::hash_block_size()

let mut poseidon_table_data: PoseidonHashTable<F> = PoseidonHashTable::default

// Feature: zktrie - MPT 해시 생성

#[cfg(feature = "zktrie")]

{

let mpt_hashes = get_storage_poseidon_witness(block);

if mpt_hashes.len() > max_hashes {

log::error!(

"poseidon max_hashes: {:?} not enough. {:?} needed by zktrie proof",

max_hashes,

mpt_hashes.len()

);

}

poseidon_table_data.fixed_inputs(&mpt_hashes);
```

```
}

// Feature: poseidon-codehash - 바이트코드 해시 생성

#[cfg(feature = "poseidon-codehash")]

{

use  crate::bytecode_circuit::bytecode_unroller::unroll_to_hash_input_default;

for  bytecode  in  block.bytecodes.values() {

if !bytecode.bytes.is_empty() {

let  unrolled_inputs =

unroll_to_hash_input_default::<F>(bytecode.bytes.iter().copied());

poseidon_table_data.stream_inputs(

&unrolled_inputs,

bytecode.bytes.len() as u64,

HASH_BLOCK_STEP_SIZE,

);

}

}

}

Self(poseidon_table_data, max_hashes)

}
```

zkTrie Witness 생성

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 253-271)

```
#[cfg(feature = "zktrie")]

fn  get_storage_poseidon_witness<F: Field>(

block: &crate::witness::Block,

) -> Vec<([F; 2], F, Option<F>)> {

use itertools::Itertools;

use mpt_circuits::{gadgets::mpt_update::hash_traces, types::Proof};

hash_traces(

&block

.mpt_updates

.proof_types

.iter()

.cloned()

.zip_eq(block.mpt_updates.smt_traces.iter().cloned())

.map(Proof::from)

.collect_vec(),

)

.into_iter()

.unique_by(|(inp, domain, hash)| {

(

inp.map(|f| f.to_bytes()),

domain.to_bytes(),

hash.to_bytes(),
```

```
)
})
.map(|(inp, domain, hash)| (inp.map(F::from), domain.into(), Some(F::from(hash))
.collect()
}
```

프로세스:

1. `block.mpt_updates` 에서 MPT 업데이트 추출
2. `proof_types` 와 `smt_traces` 를 `Proof` 로 변환
3. `hash_traces` 로 필요한 Poseidon 해시 계산
4. 중복 제거 (`unique_by`)
5. 필드 원소로 변환하여 반환

행 수 계산

min_num_rows_block 구현

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 93-186)

```
fn min_num_rows_block(block: &witness::Block) -> (usize, usize) {
// 해시 카운터 초기화
let mut path_hash_counter: HashMap<[u8; 32], usize> = Default::default();
let mut account_counter: HashMap<[u8; 32], usize> = Default::default();
let mut storage_counter: HashMap<[u8; 32], usize> = Default::default();
let mut key_counter: HashMap<[u8; 32], usize> = Default::default();
```

```
// SMT 트레이스에서 해시 수집
for smt_trace in &block.mpt_updates.smt_traces {
// 1. Account path 해시
insert(&mut path_hash_counter, smt_trace.account_path[0].root.0);
for node in &smt_trace.account_path[0].path {
insert(&mut path_hash_counter, node.value.0);
}
for node in &smt_trace.account_path[1].path {
insert(&mut path_hash_counter, node.value.0);
}
// 2. Account leaf 해시 (4개 필드)
if let Some(node) = smt_trace.account_path[0].leaf {
insert(&mut account_counter, node.value.0);
}
if let Some(node) = smt_trace.account_path[1].leaf {
insert(&mut account_counter, node.value.0);
}
// 3. Account key 해시
insert(&mut key_counter, smt_trace.account_key.0);
// 4. Storage path 해시
if let Some(path) = &smt_trace.state_path[0] {
for node in &path.path {
insert(&mut path_hash_counter, node.value.0);
```

```
}
}
if let Some(path) = &smt_trace.state_path[1] {
for node in &path.path {
insert(&mut path_hash_counter, node.value.0);
}
}
// 5. Storage leaf 해시
if let Some(node) = smt_trace.state_path[0].as_ref().and_then(|pt| pt.leaf) {
insert(&mut storage_counter, node.value.0);
}
if let Some(node) = smt_trace.state_path[1].as_ref().and_then(|pt| pt.leaf) {
insert(&mut storage_counter, node.value.0);
}
// 6. Storage key 해시
if let Some(hash) = smt_trace.state_key {
insert(&mut key_counter, hash.0);
}
}
// 중복 제거 전후 크기 계산
let prev_dedup_size = sum_count(&path_hash_counter)
+ sum_count(&key_counter)
+ sum_count(&account_counter) * 4 // Account는 4개 필드
```

```
+ sum_count(&storage_counter);

let  after_dedup_size = path_hash_counter.len()

+ key_counter.len()

+ account_counter.len() * 4

+ storage_counter.len();

// MPT 행 수 계산

let  mpt_row_num = after_dedup_size * F::hash_block_size();

// 바이트코드 행 수 계산

let  byte_row_num = block

.bytecodes

.values()

.map(|bytecode| bytecode.bytes.len() / HASH_BLOCK_STEP_SIZE + 1)

.sum::<usize>()

* F::hash_block_size();

let  total_row_num = mpt_row_num + byte_row_num;

(

total_row_num,

block.circuits_params.max_poseidon_rows.max(total_row_num),

)

}
```

계산 요소:

1. **Path 해시**: 트리 경로의 각 노드

2. **Key 해시**: Account 주소와 Storage 키

3. **Account 해시**: 계정 데이터 (4개 필드 × 계정 수)

4. **Storage 해시**: 스토리지 값

5. **Bytecode 해시**: 바이트코드 청크

회로 합성

synthesize_sub 구현

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 188-201)

```
fn synthesize_sub(
&self,
config: &Self::Config,
_challenges: &Challenges<Value<F>>,
layouter: &mut impl Layouter<F>,
) -> Result<(), Error> {
// PoseidonHashChip 생성
let chip = PoseidonHashChip::<_, HASH_BLOCK_STEP_SIZE>::construct(
config.0.clone(),
&self.0,
self.1,
);


// 회로에 witness 할당
```

```
chip.load(layouter)

}
```

프로세스:

1. `PoseidonHashChip` 생성 (hash-circuit 크레이트)
2. 설정, 해시 테이블 데이터, 최대 해시 수 전달
3. `chip.load()` 로 witness 할당

zkTrie에서의 사용

zkTrie 노드 해싱

zkTrie는 Poseidon을 사용하여 노드를 해싱합니다:

노드 타입별 해싱:

1. **Leaf Node** (계정):

```
hash = Poseidon([

field0, // [code_size | nonce]

field1, // balance

field2, // storage_root

field3, // keccak_code_hash

field4 // poseidon_code_hash

])
```

2. **Leaf Node** (스토리지):

```
hash = Poseidon([storage_value])
```

3. **Middle Node**:

```
hash = Poseidon([left_child_hash, right_child_hash])
```

경로 해싱

트리 경로를 따라 올라가면서 Poseidon 해시를 계산:

```
Root = Poseidon(
Poseidon(
Poseidon(Leaf1, Leaf2),
Poseidon(Leaf3, Leaf4)
),
...
)
```

바이트코드 해싱

Poseidon 코드 해시

Feature: `poseidon-codehash`

바이트코드를 Poseidon으로 해싱하여 zkTrie에 저장:

프로세스:

1. 바이트코드를 청크로 분할 (`HASH_BLOCK_STEP_SIZE` 바이트씩)
2. 각 청크를 필드 원소로 변환
3. Poseidon 해시 계산 (스트리밍 방식)
4. 최종 해시를 `poseidon_code_hash` 필드에 저장

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 76-88)

```
#[cfg(feature = "poseidon-codehash")]
{
use crate::bytecode_circuit::bytecode_unroller::unroll_to_hash_input_default;
for bytecode in block.bytecodes.values() {
if !bytecode.bytes.is_empty() {
let unrolled_inputs =
unroll_to_hash_input_default::<F>(bytecode.bytes.iter().copied());
poseidon_table_data.stream_inputs(
&unrolled_inputs,
bytecode.bytes.len() as u64,
HASH_BLOCK_STEP_SIZE,
);
}
}
}
```

이중 코드 해시

Scroll은 두 가지 코드 해시를 유지합니다:

1. **Keccak Code Hash**: 이더리움 호환성
2. **Poseidon Code Hash**: zkTrie 효율성

장점:

- 이더리움 네트워크와 호환
- zkTrie에서 효율적인 증명 생성
- 두 해시 간 일관성 검증

테이블 룩업

MPT Circuit 룩업

MPT Circuit은 Poseidon 테이블을 룩업하여 해시 검증:

```
// MPT Circuit에서 Poseidon 테이블 룩업
meta.lookup("mpt poseidon lookup", |meta| {
vec![
(hash_id, poseidon_table.hash_id),
(input0, poseidon_table.input0),
(input1, poseidon_table.input1),
(control, poseidon_table.control),
(domain_spec, poseidon_table.domain_spec),
]
});
```

Bytecode Circuit 룩업

Bytecode Circuit도 Poseidon 테이블을 사용:

```
// Bytecode Circuit에서 Poseidon 코드 해시 룩업
meta.lookup("bytecode poseidon lookup", |meta| {
vec![
(code_hash, poseidon_table.hash_id),
// ... 입력 필드들
]
});
```

최적화 기법

중복 제거

동일한 해시는 한 번만 계산:

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 260-268)

```
.unique_by(|(inp, domain, hash)| {
(
inp.map(|f| f.to_bytes()),
domain.to_bytes(),
hash.to_bytes(),
)
```

```
})
```

효과:

- 동일한 계정/스토리지가 여러 번 접근되어도 해시는 한 번만 계산
- 회로 크기 감소
- 증명 시간 단축

스트리밍 해싱

긴 바이트코드를 청크 단위로 처리:

```
poseidon_table_data.stream_inputs(
&unrolled_inputs,
bytecode.bytes.len() as u64,
HASH_BLOCK_STEP_SIZE,
);
```

장점:

- 메모리 효율적
- 임의 길이 입력 지원
- 병렬 처리 가능

도메인 분리

`domain_spec` 필드로 다른 용도의 해시 구분:

- Domain 0: 바이트코드 해싱
- Domain 1+: zkTrie 노드 해싱

보안:

- 서로 다른 용도의 해시가 충돌하지 않음
- 도메인 분리로 안전성 향상

회로 파라미터

행 수 설정

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 56)

```
let max_hashes = block.circuits_params.max_poseidon_rows / F::hash_block_size()
```

파라미터:

- `max_poseidon_rows` : 회로의 최대 행 수
- `hash_block_size()` : 한 해시당 필요한 행 수
- `max_hashes` : 처리 가능한 최대 해시 수

크기 추정

```
let mpt_row_num = after_dedup_size * F::hash_block_size();
let byte_row_num = (bytecode_chunks) * F::hash_block_size();
let total_row_num = mpt_row_num + byte_row_num;
```

외부 의존성

hash-circuit 크레이트

Poseidon Circuit은 `hash-circuit` 크레이트를 사용:

Cargo.toml:

```
hash-circuit.workspace = true
```

주요 타입:

- `PoseidonHashConfig` : 회로 설정
- `PoseidonHashChip` : 회로 칩
- `PoseidonHashTable` : 해시 테이블 데이터
- `Hashable` : 해시 가능한 타입 트레이트

mpt-circuits 크레이트

MPT 증명 생성에 사용:

```
use mpt_circuits::{gadgets::mpt_update::hash_traces, types::Proof};
```

테스트

Circuit 트레이트 구현

파일: `zkevm-circuits/src/poseidon_circuit.rs` (line 209-239)

```
#[cfg(any(feature = "test", test))]
impl<F: Field + Hashable> Circuit<F> for PoseidonCircuit<F> {
type Config = (PoseidonCircuitConfig<F>, Challenges);
type FloorPlanner = SimpleFloorPlanner;
```

```
type Params = ();

fn without_witnesses(&self) -> Self {
Self(Default::default(), self.1)
}

fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config {
let challenges = Challenges::construct(meta);
let poseidon_table = PoseidonTable::construct(meta);

let config = {
PoseidonCircuitConfig::new(
meta,
PoseidonCircuitConfigArgs { poseidon_table }
)
};

(config, challenges)
}

fn synthesize(
&self,
(config, challenges): Self::Config,
```

```
mut  layouter: impl Layouter<F>,
) -> Result<(), Error> {
let  challenges = challenges.values(&layouter);
self.synthesize_sub(&config, &challenges, &mut  layouter)
}
}
```

관련 문서

- MPT Circuit: zkTrie 구조와 Poseidon 사용
- Bytecode Circuit: 바이트코드 검증
- Poseidon Table: 테이블 스키마 (예정)
- zkTrie: zkTrie 구현 세부사항

요약

Poseidon Circuit은 SNARK-friendly Poseidon 해시 함수를 효율적으로 검증하는 회로입니다. zkTrie의 노드 해싱과 바이트코드 해싱에 사용되며, 중복 제거와 스트리밍 처리를 통해 최적화되어 있습니다. 외부 `hash-circuit` 크레이트를 래핑하여 구현되며, MPT Circuit과 Bytecode Circuit에서 룩업 테이블로 사용됩니다.